R wrapper strategies for FSDA-bridge: evaluation matrix

FSDA-bridge | spec 025 | branch spec-025/r-wrapper-comparison | generated 2026-10-22

Spec 025. Three ways to expose FSDA to R over the shared Python engine, each tested on the same machine against real MATLAB + FSDA. Every cell cites a log under `tools/r-compare/logs/`, produced by the script named in that row.

	Approach	Where
A	Sol 2 (Qaseem): dynamic `fsda$<name>(...)` facade, no per-routine code	`packages/fsdabridge/` (verbatim from `qaseem97/Rpackage@4f701d9`)
B	Sol 1 (Giacomo): one generated R function per routine, built from the codegen IR	`tools/codegen/gen_r.py` + `tools/codegen/tmp_spec.json` → `packages/fsdabridgeGen/`
C	Hybrid: A + `.DollarNames` fed from a JSON routine list + `fsda_help(name)`	`tools/r-compare/hybrid/` overlay, built by `tools/r-compare/make_hybrid.R` as `fsdabridgeHybrid`

Environment. macOS, R 4.6.1, reticulate 1.47.0, Python 3.12.0, MATLAB R2026a Update 2, matlabengine 26.1.12, FSDA Add-On (389 public routines in its `Contents.m` inventory). All three packages ship the same `engine.py` (`code/fsda_engine/engine.py`), so any difference comes from the R layer.

The matrix

#	Criterion	A: dynamic `$`	B: generated	C: hybrid
1	CRAN `--as-cran` (E/W/N, time)	**0 / 0 / 2**, 9 s	**0 / 0 / 1**, 9 s (83-routine dry run: 0 / 0 / 2, 10 s)	**0 / 0 / 2**, 10 s
2	Files to change for a new routine	**0**	**1 hand-edited + 4 generated**; more if FSDA has no JSON entry (true for 303 of 389)	**0** to call it; 1 data file to complete it
3	Autocompletion	`fsda$Sc` → `Score` (19 hard-coded names)	`Sc` → `Score`; `Score(y, X,` → `intercept= la= Lik= nocheck= tukey1df= nargout=`	`fsda$Sc` → 5 names (all 389 routines); no argument completion
4	`?Score`	**No** (10 Rd topics, generic `?fsda`)	**Yes**: 100-line Rd built from the IR (quality depends on the IR)	**No** Rd; `fsda_help("Score")` returns 103 lines of FSDA's own help (0.76 s, live)
5	Correctness (check_engine.R, 1e-9)	**26/26 PASS**	**13 PASS / 0 FAIL / 13 SKIP** (all 10 wrapped routines pass; 3 more cases use only `eval_m`)	**26/26 PASS**
6	Coverage (callable / 389)	**389 / 389 (100 %)**	**10 / 389 (2.6 %)** wrapped; at most 83 (21 %) from FSDA's JSON; others only through `fsda_call()`	**389 / 389 (100 %)**; all 389 completable and with help
7	Name/value options	`la`, `intercept` exact; `maxit` (not in JSON) reaches MATLAB	Same results; `maxit` only works through `...` (not completed, not documented)	Same as A
8	`mcd(Y, nargout = 2)`	list of 2 (`mcd`, `mcdr`)	identical	identical

#	Criterion	A: dynamic `$`	B: generated	C: hybrid
9	Errors	Typos caught in R; everything else is a MATLAB error	Missing argument and typos caught **in R** (plain base-R messages); everything else is a MATLAB error	Same as A
10	Namespace clashes	none (one exported object `fsda`)	none for 10 or 83 routines; **3 maskings** (`stats::supsmu`, `robustbase::Qn`, `robustbase::Sn`) if extended to all 389	none
11	Load time / tarball	2 ms / 30.2 KB	2 ms / 54.8 KB (83 routines: 2 ms / 273.5 KB)	2 ms / 35.0 KB
12	R code volume (code + roxygen)	247 + 235 hand-written	80 + 346 generated per 10 routines (664 + 2374 for 83), plus 169 runtime lines, 226-line generator and a 1,947-line hand-curated IR	A's 247 + 235, minus 23, plus 31 + 20
13	Matches pyfsda's call syntax	36/36 routine calls map 1:1 (`pyfsda.X(...)` → `fsda$X(...)`)	8/36 map to a generated function; the rest need `fsda_call()`	Same as A
14	JSON can drive a Julia façade	—	The IR is language-neutral; `gen_r.py`'s logic carries over	The routine list can replace FSDAjl's hard-coded 17-name list as is

Recommendation: C. Details are at the end.

Evidence by row

1 — CRAN check · `tools/r-compare/01_cran.sh` → `01_cran_*.log`

`R CMD build` then `R CMD check --as-cran --no-manual` on each tarball. `FSDA_LIVE` was unset, so the live tests skipped.

	Status	NOTEs	Wall time (build + check)
A	0 E, 0 W, 2 N	"New submission"; README/NEWS not checked (pandoc missing on this machine)	9 s
B	0 E, 0 W, 1 N	"New submission"	9 s
C	0 E, 0 W, 2 N	same as A	10 s
B, all 83 IR routines (`12_dryrun_full.log`)	0 E, 0 W, 2 N	adds `checkRd: tclust.Rd:93: Escaped LaTeX specials: \#`, a real gen_r.py escaping bug	10 s

All NOTEs come from the environment or a first submission. Two caveats: `--no-manual` skips the PDF manual check (no LaTeX here), and B ships **no tests** (its check has nothing to run).

Side finding for A: `NAMESPACE` contains `importFrom(utils,.DollarNames)`, but no `@importFrom` tag in `R/` produces it. Re-running roxygen on A drops that line. C's overlay adds the tag.

2 — Maintainability · `02_maintainability.sh` → `02_maintainability.log`

The test routine is `ace`, which has an FSDA JSON entry but is not in `tmp_spec.json`.

- **A:** `fsda$ace(y, X)` works with no change (`05_live_A.log`, row 7).
- **B:** add its IR record to `tmp_spec.json` (1 hand edit), regenerate, and 4 generated files differ: `DESCRIPTION`, `NAMESPACE`, `R/fsda_ace.R`, `man/ace.Rd`.
- **B, a routine with no FSDA JSON entry** (`FSRaddt`): `parse_fsda.py` emits no record, so someone has to hand-write a signature with types. The same was needed for 4 of the 10 routines here: **mahalFS, Score, mcd and logfactorial have no `functionSignatures.json` entry.** FSDA's JSON covers 86 of 389 routines; the parser keeps 83 of them.
- **C:** like A for calling. A routine added to FSDA later needs `fsda_routines.json` regenerated (1 data file) to appear in completion, and is callable even before that.

3 — Autocompletion · `offline_rows.R` → `offline_rows.log`

This uses R's own console completer (`utils:::.completeToken`), the same one the console and ESS use.

```
A  .DollarNames(fsda, "Sc"): Score                TAB "Score(y, X, ": (nothing)
B  TAB "Sc": Score                                TAB "Score(y, X, ": y= X= intercept= la= Lik=
nocheck= tukey1df= nargout=
   formals(Score): y, X, intercept, la, Lik, nocheck, tukey1df, ..., nargout
C  .DollarNames(fsda, "Sc"): Score ScoreYJ ScoreYJall ScoreYJmle ScoreYJpn
```

A completes only its 19 hard-coded names, plus names already used in the session. C reads `inst/extdata/fsda_routines.json` (389 names, written by `make_routine_list.py` from the codegen enumerator). Only B completes argument names.

4 — Documentation · `offline_rows.log`; `05_live_C.log`

- **A:** `?Score` → 0 hits. There are 10 Rd topics, all for the bridge.
- **B:** `?Score` → 1 hit. `Score.Rd` renders to 100 lines with parameters, types, defaults and see-also. The quality follows the IR: `Score`'s long description came out **empty**, and its output `outSC` has no description, because `_extract_m_prose` misses them. `logfactorial.m` has no `%% Beginning of code` marker, so its prose is empty too.
- **C:** no Rd for routines. `fsda_help("Score")` runs `help Score` in the session and returns 5,220 characters / 103 lines in 0.76 s. The first version returned MATLAB's hyperlink markup (`<strong>`, `<a href="matlab:...">`), and C now strips it. The help is always in step with the installed FSDA, but it needs a live engine and is not reachable through `?`.

5 — Correctness · `live_rows.R` via `05_live.sh` → `05_live_{A,B,C}.log`

`live_rows.R` takes the 26 case functions from `code/fsda_engine/check_engine.R` without editing them. It rebinds `fsda_call()` / `eval_m()` to each approach's own surface: A and C use `fsda[[name]](...)`; B uses the generated function, or SKIP when none was generated.

- A: `ROW5 PASS=26 FAIL=0 SKIP=0 ERROR=0`
- B: `ROW5 PASS=13 FAIL=0 SKIP=13 ERROR=0`. The 10 wrapped routines pass, at max diffs from 0 to 1.8e-15. The other 3 passes (nested struct, table, distribspec) use only `eval_m`. The 13 SKIPs are routines B does not wrap (FSRaddt, FSM, bc, …), so they show a coverage gap, not a failure.
- C: `ROW5 PASS=26 FAIL=0 SKIP=0 ERROR=0`

None of the three had a failure to classify. One more data point: `tclustIC`'s JSON types `kk` as `int16|int32`, and the R double `c(2, 3)` passes. Had B enforced JSON types in R, this correct call would have been rejected. The types are editor hints.

6 — Coverage · `05_live_{A,C}.log`, `05_live_{B,Bfull}.log`

- A, C: `ROW6 inventory=389 on_path=389` (every routine resolves through `which` in the live session, and `$` dispatches any name).
- B: `ROW6 wrappers=10`. The dry run gives `wrappers=83`, the ceiling the FSDA JSON allows today. `fsda_call()` is still exported as an escape hatch, which is approach A's mechanism without the `$` sugar.

7 — Name/value options · `05_live_*.log`

- `Score(y, X, la = LA, intercept = TRUE)` → max |Score – gold| = 0 in A, B and C. `intercept = FALSE` changes the result in all three, so the logical option crosses.
- An option missing from the JSON: `ace`'s FSDA JSON entry declares only `y` and `X`, yet `ace.m` takes `maxit`, `nterm`, `delrsq`, `l` and `w`. `ace(y, X, maxit = 3)` versus the default gives max |Δty| = 1.23e-2, so the option reaches MATLAB in A, C and B-full. In B, `formals(ace)` is `y, X, ..., nargout`: `maxit` works only because the generator emits `...`. It is neither completed nor documented. A generator without `...` would turn this into "unused argument".
- `Score(y, X, notAnOption = 1)`: all three forward it, and FSDA's `aux.chkoptions` rejects it (MatlabExecutionError, so bad input, not a bridge gap).

8 — Multiple outputs · `05_live_*.log`

`mcd(Y, nargout = 2, plots = 0, msg = 0)` after `rng(0)` gives the same result in all three: a list of 2, classes `mcd` / `mcdr`, `RAW$loc = [−0.639089 −0.216581 0.210577]`, and REW with 7 fields.

9 — Errors · `05_live_{A,B}.log` (C behaves as A)

Input	A / C: where raised and message	B: where raised and message
`mahalFS("abc", MU, SIGMA)`	MATLAB: `MatlabExecutionError … mahalFS.m, line 64 … Non-singleton dimensions`	same
`Score(y)` (X missing)	MATLAB: `… Score.m, line 139 … Not enough input arguments`	**R**: `argument "X" is missing, with no default`
typo `Scroe`	**R** (facade name check): `FSDA routine 'Scroe' was not found on the MATLAB path. Check the spelling…`	**R**: `could not find function "Scroe"`
`mahalFS(list(1, 2), …)`	**no error**: the list crosses as a row vector (0.5714)	same
`Score(y, X, la = "x")`	**no error**: returns `NaN NaN` with MATLAB singular-matrix warnings	same

The MATLAB errors are readable: file, line and FSDA's message, followed by reticulate's `py_last_error()` hint. They do include the absolute FSDA install path. The two silent cases show that none of the approaches checks types in R. B could, from the JSON, but row 5 shows the JSON types are too loose to enforce.

10 — Namespace clashes · `offline_rows.log`

In a fresh R with `library(stats); library(robustbase); library(<pkg>)`, `conflicts(detail = TRUE)` lists nothing for any of the packages, and `conflicts()` totals 4 in every case (base/stats pre-existing). A and C export one object, `fsda`, plus bridge helpers. B exports one function per routine. Its 10 and 83 names do not collide with base, stats, utils, graphics, grDevices, methods, robustbase, MASS, cluster, Matrix, lattice, nlme,

mgcv or survival. The full 389-name inventory **would mask `stats::supsmu`, `robustbase::Qn` and `robustbase::Sn`**.

11 — Load time and size · `offline_rows.log`

`library()` takes a 2 ms median (n = 10, fresh R each time) for all three, and 2 ms for the 83-routine B as well. None loads reticulate at attach time: it is deferred to the first call (+0.28 s). The MATLAB boot is the same for all three, about 4.8 s on this machine (`session boot` in `05_live_*.log`).

	tarball	installed
A	30,223 B	92 KB
B (10)	54,765 B	126 KB
B (83, dry run)	273,501 B	—
C	34,995 B	139 KB (includes the 42 KB routine list)

12 — Code volume · `12_loc.py` → `12_loc.log; 12_dryrun_full.log`

Non-blank lines, split as code / roxygen / comment:

- **A:** `R/` 247 / 235 / 84; tests 291 code lines.
- **B:** 10 wrappers 80 / 346 / 10; call path 9; runtime copied from A 169 / 148 / 40. `gen_r.py` is 226 lines. `tmp_spec.json` is 1,947 lines (100 KB), and 4 of its 10 records needed hand-written signatures. Full dry run (83 routines): 664 code + 2,374 roxygen R lines, 5,602 Rd lines, generated in 0.012 s plus 2.3 s of roxygen. The whole toolbox parses in 0.1 s with 1,169 warnings.
- **C:** A minus the 23-line hard-coded completion block, plus overlay `R/` 31 / 20 / 9, overlay tests 19, `make_hybrid.R` 38, and `make_routine_list.py` 42.

13 — Consistency with pyfsda · `13_consistency.py` → `13_consistency.log`

This scans every `pyfsda.<name>(...)` call in `packages/pyfsda/examples/*.py` and its README: 49 distinct calls, 36 of them routine calls and 13 pyfsda helpers (`load`, `fetch`, `clear`, `to_dataframe`, …).

- **A/C:** 36/36 map one-to-one. `pyfsda.Score(y, X, la=la, intercept=True)` becomes `fsda$Score(y, X, la = la, intercept = TRUE)`, and `nargout` / `echo_output` / `options` mean the same thing.
- **B:** 8/36 map to a generated function; the other 28 need `fsda_call(NULL, "X", ...)`.
- **All three:** 5 calls use pyfsda-only keywords (`frames=`, `store=`), and none of the 13 helpers exists in R. The R packages bundle the older `code/fsda_engine/engine.py`, not pyfsda's. That gap is the same whichever approach is chosen.

14 — Julia reuse (note)

Yes, either JSON can drive a Julia façade. `packages/FSDAjl/src/FSDABridge.jl` already generates its façade with `@eval` over a **hard-coded list of 17 routines**. Swapping that list for `fsda_routines.json` (C's list) gives all 389 routines with no other change. Using the IR (B's input) as well adds per-routine docstrings and keyword names for the 83 routines that have one. The IR keeps MATLAB types, so `gen_r.py`'s type-to-prose and NULL-drop logic port directly.

A's source says Julia "cannot" intercept unknown names. That holds for a module, but a Julia struct with `Base.getproperty` (`fsda.Score(...)`) can, which mirrors R's `$` if a list-free Julia façade is wanted.

Recommendation

Adopt C. It keeps all of A's strengths: 26/26 at 1e-9, 389/389 routines, no per-routine maintenance, and pyfsda's call syntax. It fixes A's two weak points, discoverability (19 hard-coded names → the full inventory) and in-R documentation (`fsda_help`). The cost is 51 lines, one `jsonlite` import and a 42 KB data file, with no extra CRAN findings.

Why not B as the primary surface. B gives the best R-native experience for the routines it covers: argument completion, `?Score`, and argument-missing errors raised in R. But its coverage depends on FSDA's `functionSignatures.json`, which covers **86 of 389 routines and leaves out mahalFS, Score, mcd and logfactorial**. Closing that gap means hand-writing typed signatures (Layer 0 is not ours to edit) or getting them into FSDA upstream. B also needs regeneration and a release for every new routine. At full scale it would mask 3 functions from stats/robustbase, and the generator still has rough edges: the `\#` Rd escape, empty prose for Score and logfactorial, and options missing from the JSON that are reachable only through `...`.

Keep B's generator as a documentation path, not a call path. Run `gen_r.py`'s Rd half over the 83 IR routines to add `?Score`-style pages to C, while dispatch stays `fsda$`. That gives row 4's "Yes" where the IR is good, without the coverage ceiling. Separately, a small upstream PR adding JSON entries for the most-used routines (mahalFS, Score, mcd, LXS, FSRaddt, …) would raise the ceiling for every consumer: R, Julia and MATLAB's own editor.

Follow-ups found along the way (out of scope here):

- A's hand-edited `NAMESPACE` (row 1).
- `_extract_m_prose` misses Score's long description and logfactorial's preamble (row 4).
- The `\#` escape in `gen_r.py` (row 1).
- The R engine lags pyfsda's (`frames`, `store`) (row 13).